Advanced Computer Architecture - Project
3D scanning using depth camera images

Jonas Roels & Niels Van Bossche

May 10, 2024



Class: MEIsw

Contents

1 Introduction **1**

2 Noise filtering **1**

3 Transforming images to a point cloud **3**

4 Reducing the number of points and cropping the object **4**

5 Mesh construction **5**

6 Timing **7**

7 Conclusion and reflection **8**

1 Introduction

For our master thesis we use the Magic Leap 2 AR glasses to train maintenace technicians. These glasses are equipped with a large number of sensors, including a depth camera. This camera operates based on infrared light, initially taking a baseline measurement, after which the glasses emit a powerful infrared flash to illuminate a second image. Based on the difference between the values from both images, a depth map is constructed [1]. An example of such a depth map can be seen in Figure 2a.

By combining multiple depth maps with the position and rotation of the glasses during the creation of each depth map, we can build a point cloud. From this point cloud, we can then construct a 3D model. However, there are many challenges associated with this process. One initial challenge we encountered is filtering out the noise visible in Figure 2a. This noise is due to the limited range of the depth camera (only 0.9 meters in short-range mode), but also reflective and transparent objects such as windows and computer screens.

Another challenge relates to the large number of points in the point cloud. Each image has a resolution of 544 by 480 pixels, resulting in 261,120 points per image used in the scan.

A third challenge is how to separate the object from its surroundings. The next problem we encountered involves merging the different images. Due to a drift in the position of the Magic Leap, the points between different images deviate by several centimeters. This deviation is also visible in Figure 1a. For the continuation of this proof of concept, we manually aligned the different images. An example of an aligned point cloud can be seen in Figure 1b.

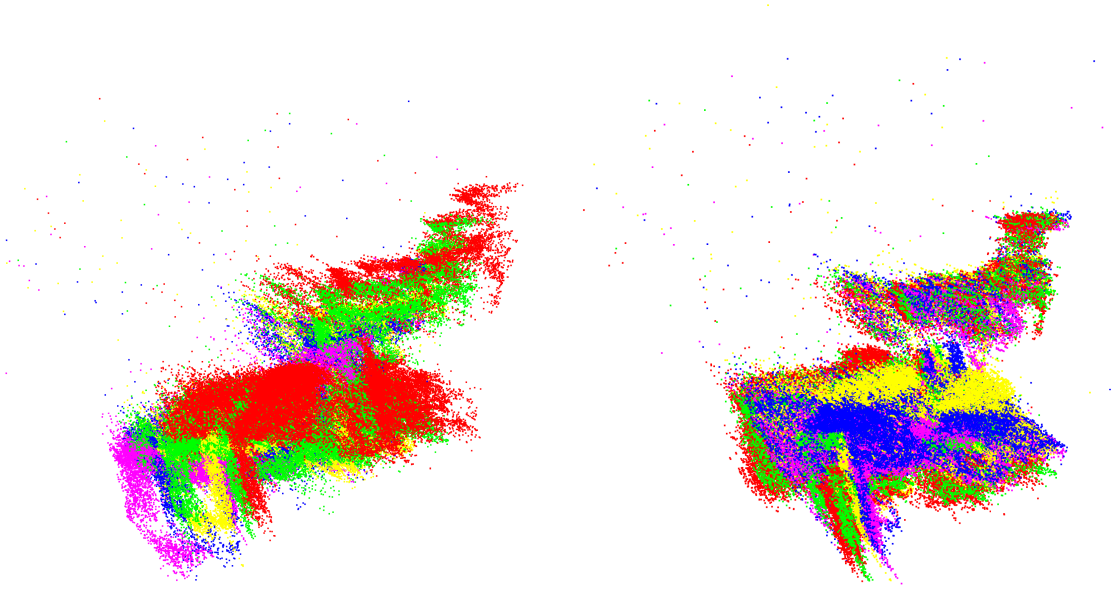
Finally, there is the challenge of effectively constructing a mesh from the point cloud.

2 Noise filtering

To remove noise from the image, we calculate the convolution of the Sobel operator in both the X and Y directions (using the convolution kernels shown in Equation 1) with the image. From the resulting 2 values, we determine the magnitude of the gradient, which can also be represented as an image. An example of this is shown in Figure 2b. This yields a large value for regions in the image with a large difference in value between adjacent pixels, which is the case in areas with noise but also at the edges of objects, potentially reducing the resolution of our scans in these areas [2].

To effectively filter out the noise, we repeat the same procedure on the gradient images. We then mask the original image, considering only points where the magnitude in both gradients is lower than a certain threshold value (chosen as 0.30 for the example) during the construction of the point cloud. The result and mask generated by this operation are shown in Figures 2d and 2c, and the resulting point clouds without and with noise removal are shown in Figures 3a and 3b.

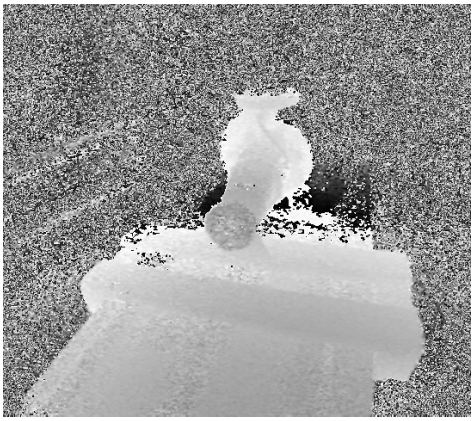
$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \tag{1}$$



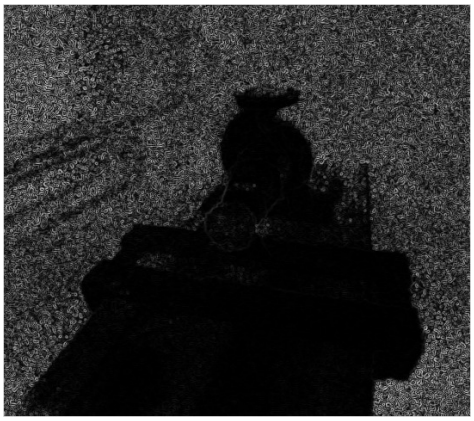
(a) Raw point cloud

(b) Manualy aligned point cloud

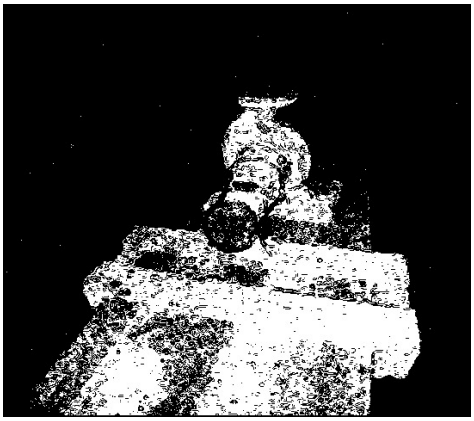
Figure 1: Examples of multi-image point clouds



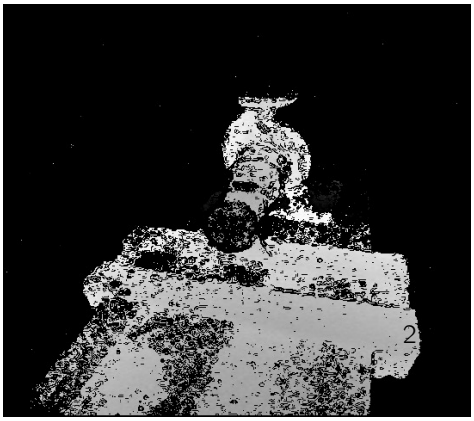
(a) Raw depth image from the depth camera of the Magic Leap 2



(b) Gradient of the Sobel operator and the mask generated from it

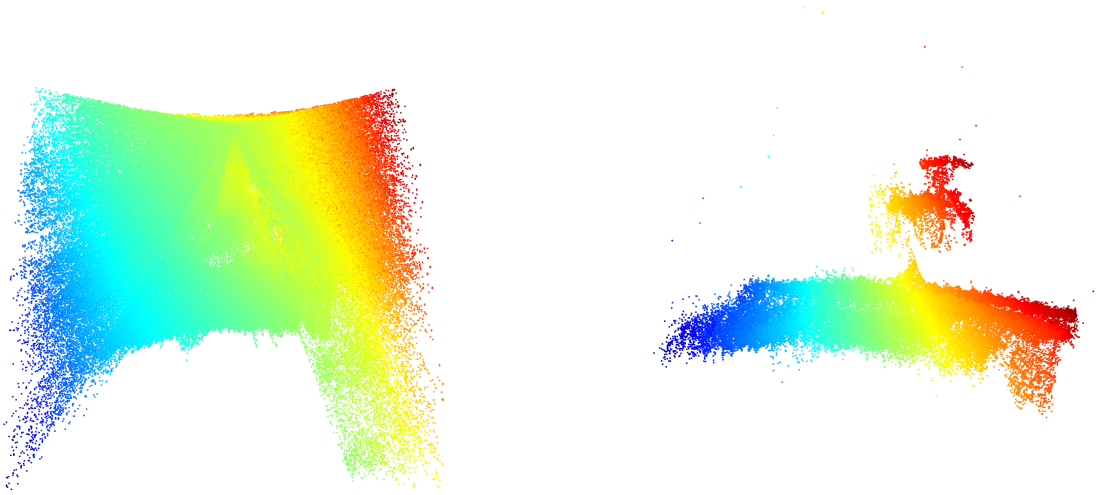


(c) Mask generated based on the gradients of the Sobel operators



(d) Masked depth image from the depth camera of the Magic Leap 2

Figure 2: Examples of depth images



(a) Raw point cloud

(b) Filtered point cloud

Figure 3: Examples of point clouds

3 Transforming images to a point cloud

The value of each pixel represents the distance from the camera to the intersected object, expressed as a 32-bit floating-point number. From the pixel's location in the frame, i.e., its row and column, we can determine the angle of the ray emanating from the camera when combined with the known field of view (FoV). This FoV is 88.69 degrees diagonally, or equivalently, 66.50 and 58.68 degrees horizontally and vertically, respectively. The points are represented in spherical coordinates.

These spherical coordinates are then translated into Cartesian coordinates. This can be done quite simply using Formulas 2, 3, and 4, where θ represents the azimuthal angle, ϕ the polar angle, and r the length of the ray. It is important to consider the left-handed coordinate system of Unity. For a visual representation, refer to Figure 4 [3].

$$x = r \cdot \sin(\theta) \cdot \cos(\phi) \tag{2}$$

$$y = r \cdot \sin(\phi) \tag{3}$$

$$z = r \cdot \cos(\theta) \cdot \sin(\phi) \tag{4}$$

Next, the rotation of the camera needs to be undone. This rotation is represented as an Euler rotation in the order Z, X, Y. To reverse this rotation, the points must be rotated by the negative of these angles, in reverse order. Once again, it is important to consider the left-handed coordinate system. The used rotation matrices can be found in Equation 5. Finally, the position of the camera is added to the coordinate values of the points as an offset.

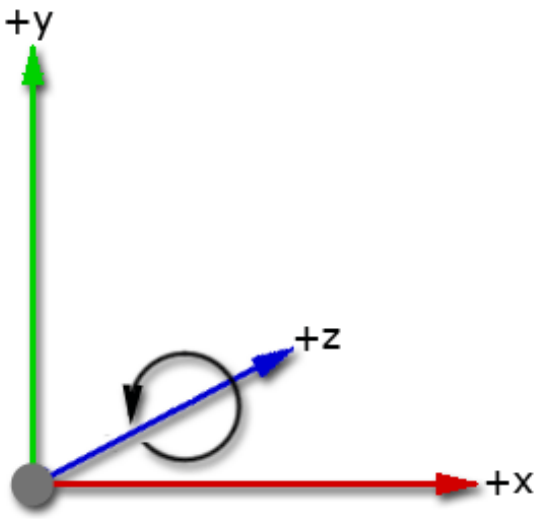


Figure 4: Left-handed coordinate system [3]

$$R_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & cos(\theta) & sin(\theta) \\ 0 & -sin(\theta) & cos(\theta) \end{bmatrix} R_y = \begin{bmatrix} cos(\theta) & 0 & -sin(\theta) \\ 0 & 1 & 0 \\ sin(\theta) & 0 & cos(\theta) \end{bmatrix} R_z = \begin{bmatrix} cos(\theta) & sin(\theta) & 0 \\ -sin(\theta) & cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (5)$$

4 Reducing the number of points and cropping the object

By separating the object from its surroundings, we already greatly reduce the number of points in the point cloud. The first step in this process is to establish a convex hull from the position data of each capture. Assuming that during each capture the object was entirely in view, we can remove points that fall outside the field of view of one or more images.

The results of this convex reduction step can be seen in Figure 5. Here red points are removed from the point cloud while green points are kept. We see that the misalignment of the scans causes false positives.

Then to separate the object from the table on which it is located, we can use algorithms such as RANSAC, Hough transformation, or region-growing based algorithms [4] [5]. To detect the correct plane, only planes that are nearly parallel to the XZ-plane need to be considered.

Finally we use the user-defined (estimated) dimensions of the object to further trim points. We do this by determining the centroid of all points. We use this point as the center for the ellipsoid with the aforementioned dimensions plus a safety margin. Furthermore, one could experiment with performing this procedure multiple times with progressively decreasing margin, as ideally, the centroid gets closer to the effective centroid of the object.

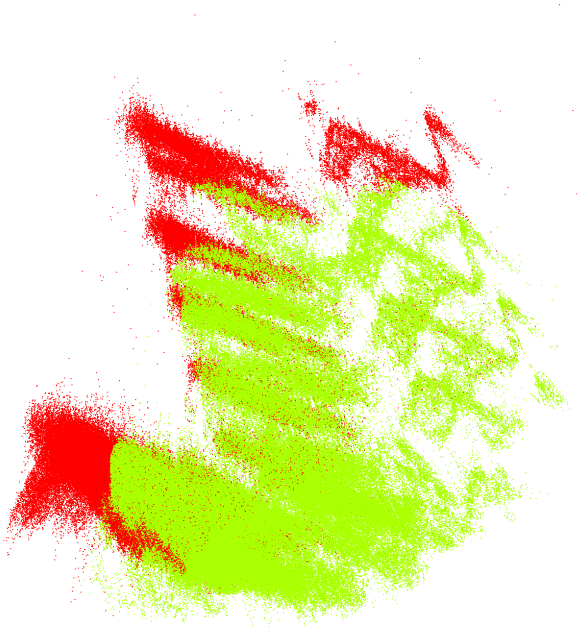


Figure 5: Example of a convex reduced point cloud

5 Mesh construction

In the literature, we find several methods for reconstructing point clouds. Traditionally, algorithms such as:

- 3D alpha-shapes reconciliation [6]
- Ball-pivoting algorithm (BPA) for surface reconstruction [7]
- Poisson Surface Reconstruction [8] [9]
- Advancing Front Surface Reconstruction [10]
- Scale-Space Surface Reconstruction [11]

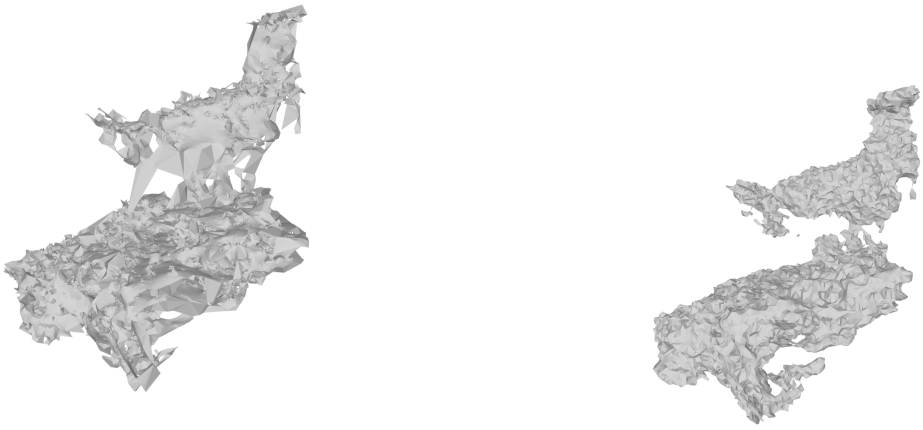
These algorithms are also found integrated into the CGAL and Open3D libraries [12] [13]. But we also find modern approaches like Point2Mesh, which is based on a Convolutional Neural Network (CNN) by Hanocka et al. This system also uses CUDA for GPU acceleration under the hood [14].

However, these approaches have a common shortcoming. For the best results, each of these algorithms expects a structured point cloud, which ours is anything but. However, there is also Vis2Mesh, developed by Song et al. This algorithm is specifically designed for mesh reconstruction from unstructured point clouds of large environments. This approach also utilizes various neural networks [15].

To explore mesh reconstruction algorithms, we utilized the CGAL Polyhedron 3 Demo tool for its simplicity and functionality. Figure 6a showcases a mesh generated through the scale-space

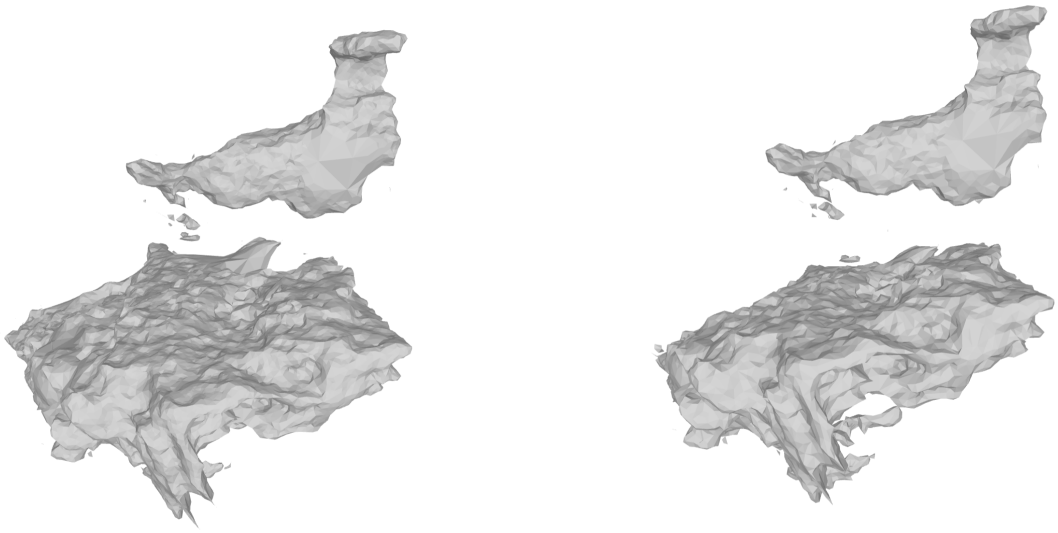
process. The resulting mesh is far from perfect. We achieved superior results with alpha shapes reconstruction, depicted in Figure 6b. Before applying these algorithms, outlier removal was conducted on the point cloud represented in Figure 1b.

Using the same tool, we further processed the mesh by using angle and area smoothing. The result of this operation can be found in Figure 7a. Lastly we reduced the number of triangles in the mesh using a simplification algorithm. The final result can be found in Figure 7b.



(a) CGAL Scale space reconstruction (b) CGAL Alpha shapes reconstruction

Figure 6: Examples of meshes constructed from the point cloud



(a) Smoothed mesh (b) Smoothed and simplified mesh

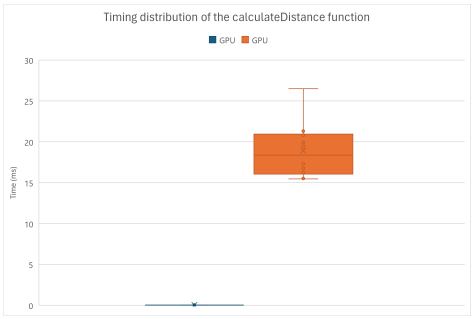
Figure 7: Examples of reconstructed meshes with extra processing

6 Timing

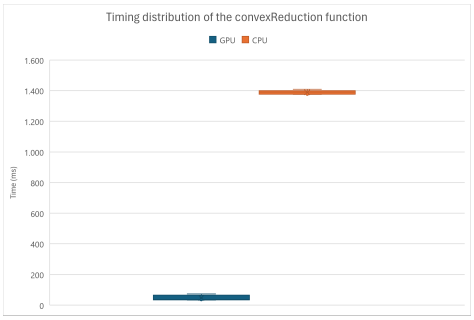
In Figures 8a, 8b and 8c and Table 1 we see the results of the timing of our program. Here we notice that the GPU is considerably faster at executing the calculateDistance and convexReduction functions within our program. The total execution time is lower but only slightly. This is due to the fact that saving the almost 600 000 points to a file takes a considerable amount of time regardless of implementation. If we disregard this time we notice that the GPU only requires on average about half as much time as the CPU. For these tests we used a sample set of fifteen images.

	CalculateDistance (ms)	**ConvexReduction (ms)**	**TotalTime (ms)**	**SaveTime (ms)**	**NoSaveTime (ms)**
GPU	0,038	47	8958	7446	1512
CPU	19	1388	10500	7365	3135

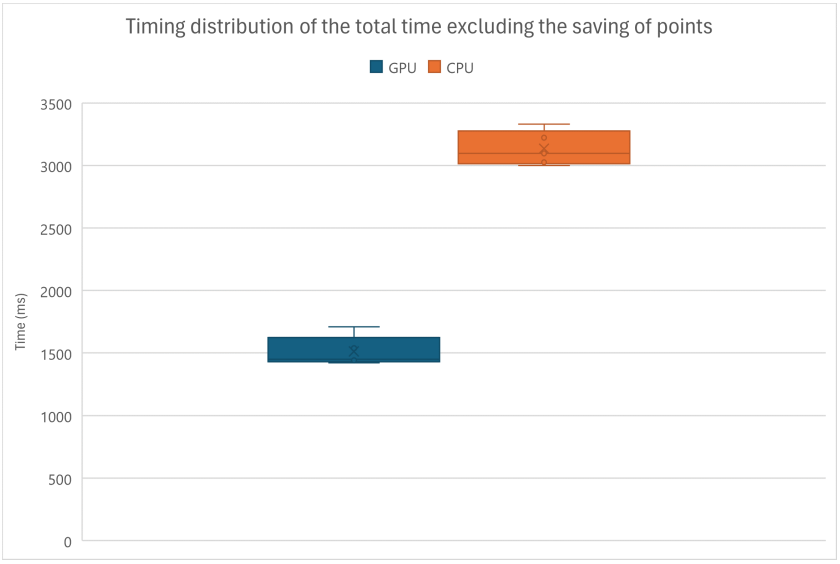
Table 1: Averages of the timing of different parts of the program



(a) Timing of the calculateDistance function



(b) Timing of the convexReduction function



(c) Total execution time without saving the points to a file

Figure 8: Timing distributions on CPU and GPU

7 Conclusion and reflection

The method presented offers a viable means to generate meshes. However, it is not without its shortcomings. Mainly the challenge of aligning multiple images accurately. This limitation impedes the attainment of precise results, restricting the number of images that can be effectively combined. Manual alignment introduces inherent imperfections, resulting in wider discrepancies as more images are integrated. Conversely, employing a smaller set of images leads to noticeable gaps in the resulting point cloud, consequently manifesting as distortions or voids in the reconstructed mesh.

To render this method practical for broader application, a streamlined approach is imperative. Relying on external tools like Open3D for visualization and CGAL for cleanup and reconstruction introduces unnecessary complexity and inefficiency.

References

[1] Sensors | MagicLeap developer documentation. [Online]. Available: https://developer-docs.magicleap.cloud/docs/device/hardware/sensors/

[2] R. Fisher, S. Perkins, A. Walker, and E. Wolfart. Feature detectors - sobel edge detector. [Online]. Available: https://homepages.inf.ed.ac.uk/rbf/HIPR2/sobel.htm

[3] U. Technologies. Unity - manual: Rotation and orientation in unity. [Online]. Available: https://docs.unity3d.com/Manual/QuaternionAndEulerRotationsInUnity.html

[4] J. Xu, Q. Xie, H. Chen, and J. Wang, "Real-time plane detection with consistency from point cloud sequences," *Sensors*, vol. 21, no. 1, 2021. [Online]. Available: https://www.mdpi.com/1424-8220/21/1/140

[5] M. Yang and W. Förstner, "Plane detection in point cloud data," University of Bonn, WorkingPaper, 2010.

[6] H. Edelsbrunner and E. Mücke, "Three-dimensional alpha shapes," 1994.

[7] F. Bernardini, J. Mittleman, H. Rushmeier, C. Silva, and G. Taubin, "The ball-pivoting algorithm for surface reconstruction," *IEEE Transactions on Visualization and Computer Graphics*, vol. 5, no. 4, pp. 349–359, 1999.

[8] M. Kazhdan, M. Bolitho, and H. Hoppe, "Poisson Surface Reconstruction," in *Symposium on Geometry Processing*, A. Sheffer and K. Polthier, Eds. The Eurographics Association, 2006.

[9] P. Alliez, L. Saboret, and G. Guennebaud, "Poisson surface reconstruction," in *CGAL User and Reference Manual*, 5.6.1 ed. CGAL Editorial Board, 2024. [Online]. Available: https://doc.cgal.org/5.6.1/Manual/packages.html#PkgPoissonSurfaceReconstruction3

[10] T. K. F. Da and D. Cohen-Steiner, "Advancing front surface reconstruction," in *CGAL User and Reference Manual*, 5.6.1 ed. CGAL Editorial Board, 2024. [Online]. Available: https://doc.cgal.org/5.6.1/Manual/packages.html#PkgAdvancingFrontSurfaceReconstruction

[11] T. van Lankveld, "Scale-space surface reconstruction," in *CGAL User and Reference Manual*, 5.6.1 ed. CGAL Editorial Board, 2024. [Online]. Available: https://doc.cgal.org/5.6.1/Manual/packages.html#PkgScaleSpaceReconstruction3

[12] The CGAL Project, *CGAL User and Reference Manual*, 5.6.1 ed. CGAL Editorial Board, 2024. [Online]. Available: https://doc.cgal.org/5.6.1/Manual/packages.html

[13] Q.-Y. Zhou, J. Park, and V. Koltun, "Open3D: A modern library for 3D data processing," *arXiv:1801.09847*, 2018.

[14] R. Hanocka, G. Metzer, R. Giryes, and D. Cohen-Or, "Point2mesh: A self-prior for deformable meshes," *ACM Trans. Graph.*, vol. 39, no. 4, 2020. [Online]. Available: https://doi.org/10.1145/3386569.3392415

[15] S. Song, Z. Cui, and R. Qin, "Vis2mesh: Efficient mesh reconstruction from unstructured point clouds of large scenes with learned virtual view visibility," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2021, pp. 6514–6524.